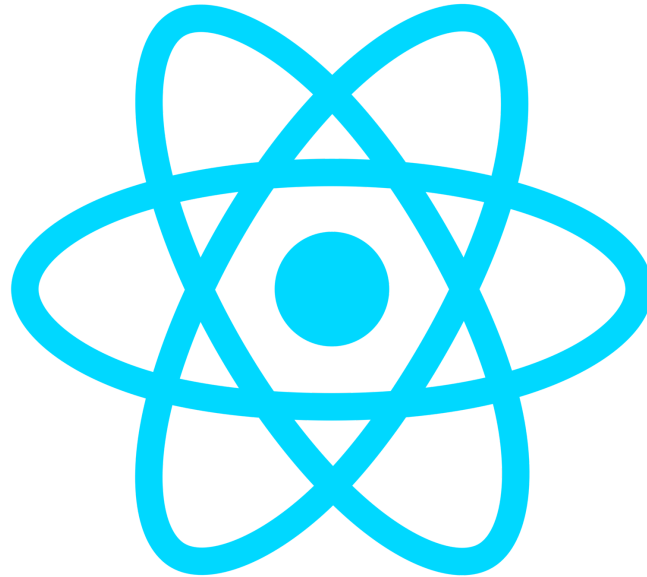


React



Une bibliothèque JavaScript pour créer des
interfaces utilisateurs

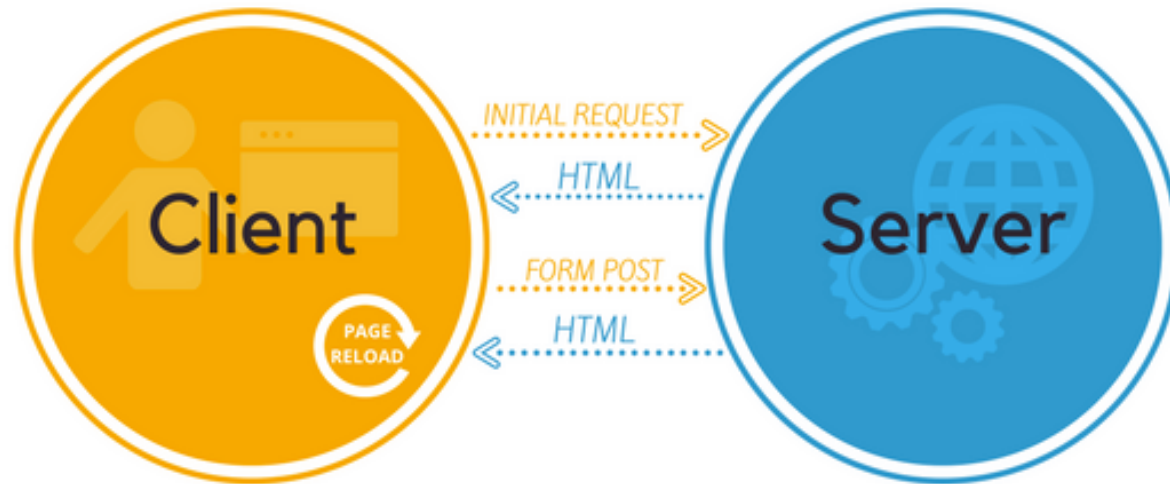
Formateur: Fabio Ginja

@Ambient-IT

Introduction à React

Multi Page Application

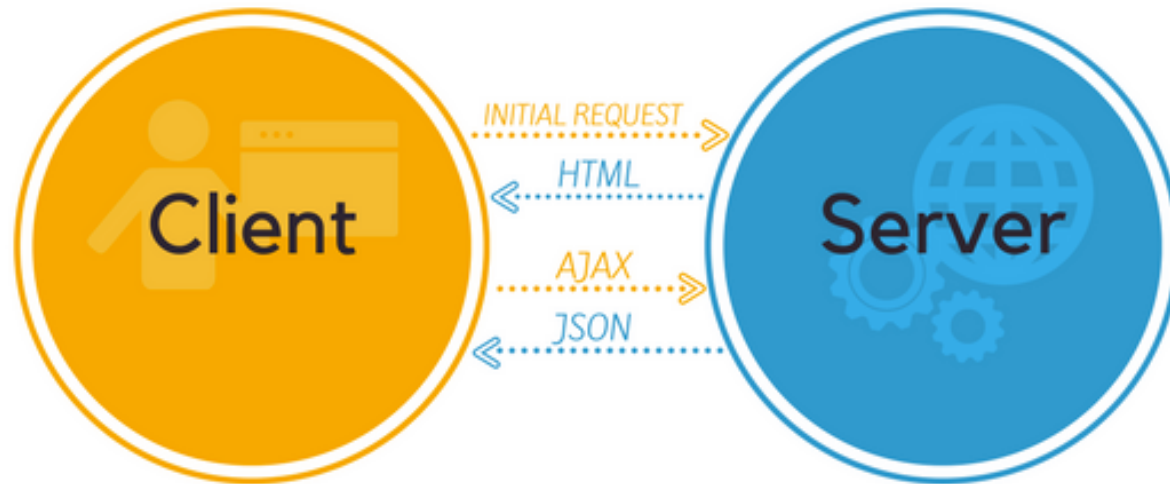
Traditional page lifecycle



Chaque changement dans notre page web force un rafraîchissement de la page, et donc le chargement d'une nouvelle page html. Application centrée **serveur**.

Single Page Application

SPA lifecycle



Chaque changement provoque un appel AJAX au serveur. Ce dernier renvoie des données et ne met à jour que certaines parties de la page. Application centrée **client**.

Frameworks Javascript

- Vue.js - Alibaba, Tencent, Baidu
- Angular - Forbes, Xbox, Parrot
- React - Facebook, Netflix, Airbnb, Slack
- Svelte - NY Times
- Ember.js - LinkedIn
- Polymer - Youtube, Google Play Music, Coca-Cola
- Backbone.js - Reddit, Amazon
- Aurelia - seloger
- Meteor - SFR
- Mithril - Vimeo, Nike

Les trois frameworks les plus populaires restent Angular, React et Vue.js.

React - À base de composants

"Définissez des vues simples pour chaque état de votre application, et lorsque vos données changeront, React mettra à jour, de façon optimale, juste les composants qui en auront besoin."

"Créez des composants autonomes qui maintiennent leur propre état, puis assemblez-les pour créer des interfaces utilisateurs complexes."

Exemple de composant

Un *Component* décrit l'HTML qui sera produit en fonction des paramètres qui lui auront été fournis.

```
1 import React from 'react'
2
3 const Card = ({name, description}) => {
4   return (
5     <div>
6       <span>Product: {name}</span>
7       <p>Description: {description}</p>
8     </div>
9   )
10 }
11
12 export default Card
```

Lorsqu'un des paramètres change (name ou description), l'HTML sera mis à jour automatiquement.

Attention, la partie à l'intérieur du return n'est pas de l'HTML, mais du JSX... On va y revenir.

Hiérarchie de composants

Chaque component peut faire appel à d'autres components. On construit ainsi notre "arbre" de components.

```
1 import React from 'react'
2 import Card from './Card'
3
4 const Parent = () => {
5   return (
6     <div>
7       <Card name="Game of thrones" description="Greats Books" />
8       <Card name="The Lion King" description="Super movie" />
9     </div>
10   )
11 }
12
13 export default Parent
```


React et Virtual DOM

Le Document Object Model (ou DOM) fournit une représentation structurée du document sous forme d'un arbre et définit la façon dont la structure peut être manipulée.

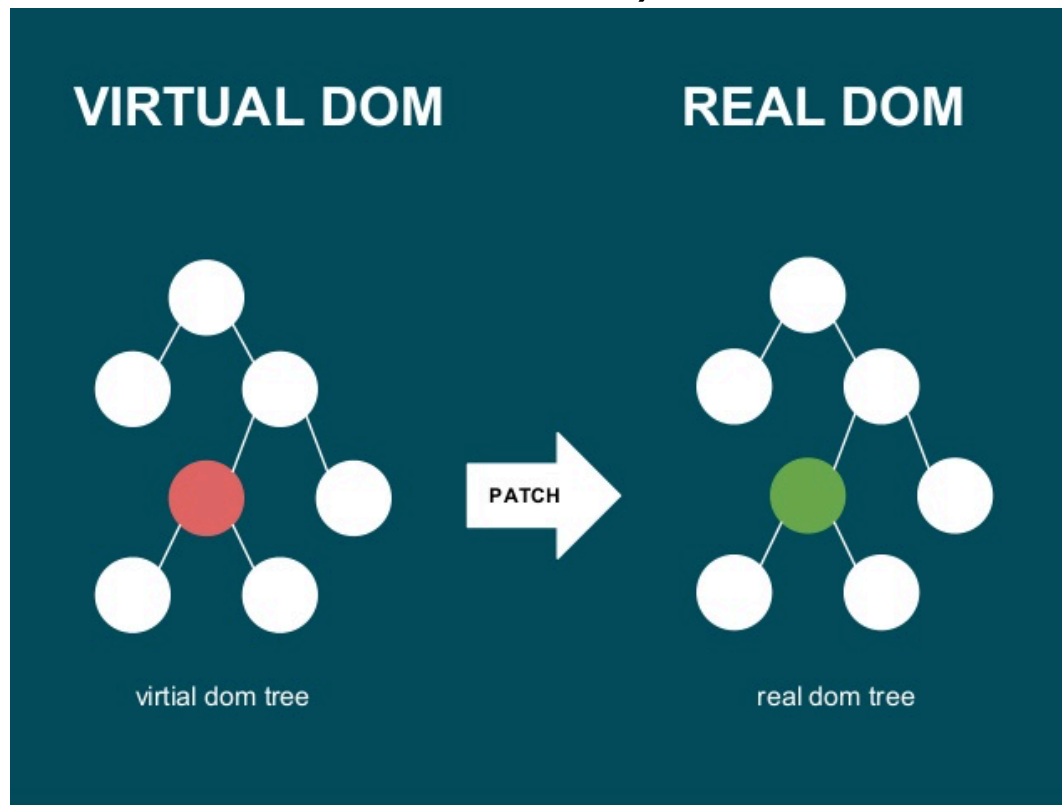
Le DOM peut être modifié en javascript comme suit:

```
document.getElementsByTagName('p')[0].innerHTML = "Injected content in first  
<p>"
```

Cependant chaque accès au DOM est coûteux... La solution de React est de garder une représentation interne du DOM: le Virtual DOM.

React et Virtual DOM

React met à jour en permanence le Virtual DOM mais n'applique que les changements vraiment nécessaires au DOM (React Fiber Reconciliation).



Créer notre première App

Avec Vite

Pour créer notre application React, on va utiliser l'outil **Vitejs** au lieu de l'outil CRA:

```
npm create vite@latest my-react-app -- --template react-ts
```



Il est également possible d'utiliser create-react-app, mais cette option n'est aujourd'hui plus recommandée car moins performante.

Documentation: <https://github.com/facebook/create-react-app>

Point d'entrée

Le point d'entrée sera notre *index.ts*. Ce dernier va injecter dans dans notre *index.html* dans la *div* ayant l'id *root* le contenu de notre *App.ts*

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App';

const root = ReactDOM.createRoot(
  document.getElementById('root') as HTMLElement
);

root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

TypeScript avec ESLint

Il faudra installer l'extension VSCode Eslint afin de bénéficier des warnings en live sur nos fichiers.

ESLint avec Prettier

Enfin, on va pouvoir installer Prettier afin de garder le même formatage du code au sein de notre projet:

```
npm install -D prettier eslint-config-prettier eslint-plugin-prettier
```

Puis créer un fichier de configuration pour Prettier:

```
touch .prettierrc
```

Avec nos différentes règles:

```
{  
  "semi": false,  
  "tabWidth": 2,  
  "printWidth": 100,  
  "singleQuote": true,  
  "trailingComma": "all",  
  "jsxSingleQuote": true,  
  "bracketSpacing": true  
}
```

Et on ajoute dans notre *package.json*:

```
{  
  "scripts": {  
    "lint": "eslint .",  
    "lint:fix": "eslint --fix .",  
    "format": "prettier --write . --config ./.prettierrc"  
  }  
}
```

ESLint avec Prettier

Il faudra ajouter les propriétés au fichier *eslint.config.js* comme suit pour ajouter le plugin Prettier:

```
import reactRefresh from 'eslint-plugin-react-refresh'
import tseslint from 'typescript-eslint'
import { globalIgnores } from 'eslint/config'
import eslintPluginPrettier from 'eslint-plugin-prettier'

export default tseslint.config([
  globalIgnores(['dist']),
  {
    files: ['**/*.ts,tsx'],
    plugins: {
      prettier: eslintPluginPrettier,
    },
    extends: [...],
    languageOptions: {...},
    rules: {
      'prettier/prettier': ['warn'],
    },
  },
])
```


Absolute Path

Plutôt que d'importer nos composants depuis un chemin relatif, on peut utiliser des chemins absolus:

```
// On écrira alors:  
import Loader from '@components/Loader';  
//Plutôt que  
import Loader from '../../../components/Loader';
```

Pour ce faire il faudra dans le fichier *tsconfig.json* ajouter:

```
"paths": {  
  "@components/*": [".src/components/*"]  
}
```

Installer la librairie: `npm i -D vite-tsconfig-paths`

Et ajouter le plugin dans notre *vite.config.ts*:

```
1 import { defineConfig } from 'vite';  
2 import react from '@vitejs/plugin-react';  
3  
4 // https://vitejs.dev/config/  
5 export default defineConfig({  
6   plugins: [react()],  
7 });
```

JSX

JSX

Le JSX, pour JavaScript & XML, est une extension à JavaScript proche du XML.

Le JSX produit des "éléments" ou composants React.

```
const name = 'Fabio'
const element = (<h1>Bonjour, {name}</h1>)

const root = ReactDOM.createRoot(
  document.getElementById('root') as HTMLElement
);

root.render(element);
```

Le JSX n'est pas obligatoire pour développer sous React (car il est ensuite compilé par React et devient une simple fonction JS). Il est cependant fortement recommandé car beaucoup plus lisible.

C'est un "syntactic sugar".

JSX - createElement

Ces deux versions sont toutes deux équivalentes:

```
1  const Button = ({name}) => {  
2    return <button>Send to {name}</button>  
3  }  
4  
5  root.render(  
6    <Button name="Fabio" />  
7  )
```

```
1  const Button = ({name}) => {  
2    return React.createElement('button', {name}, `Send to ${name}`)  
3  }  
4  
5  root.render(  
6    React.createElement(Button, {name: 'Fabio'}, null)  
7  )
```

Les balises JSX peuvent être soit des balises HTML, soit des composants que l'on a créés. Les composants que l'on crée commencent **toujours** par une **majuscule**.

Règles: un seul nœud

On ne peut retourner qu'un seul nœud/qu'une seule racine.

```
1 import React from "react"
2 import UserCard from "../Usercard"
3 const Profile = () => {
4   return (
5     <h2>Profil</h2>
6     <UserCard id={1} name="Fabio"></UserCard>
7   )
8 }
9 // Ne va pas compiler car deux éléments sont retournés
```

Il faudra écrire:

```
1 import React from "react"
2 import UserCard from "../Usercard"
3 const Profile = () => {
4   return (
5     <div>
6       <h2>Profil</h2>
7       <UserCard id={1} name="Fabio"></UserCard>
8     </div>
9   )
10 }
11 // Pas de problème ici
```

JSX

Pour renvoyer les deux éléments, sans rajouter de parent englobant les deux, on peut utiliser le *Fragment*

```
1 import React from "react"
2 import UserCard from "../Usercard"
3 const Profile = () => {
4   return (
5     <React.Fragment>
6       <h2>Profil</h2>
7       <UserCard id={1} name="Fabio"></UserCard>
8     </React.Fragment>
9   )
10 }
```

Ou sa forme raccourcie:

```
1 import React from "react"
2 import UserCard from "../Usercard"
3 const Profile = () => {
4   return (
5     <>
6       <h2>Profil</h2>
7       <UserCard id={1} name="Fabio"></UserCard>
8     </>
9   )
10 }
```

JSX

Les balises n'ayant pas de contenu/d'enfants peuvent être *self-closing* (auto-fermantes).

```
<div>
  <span></span>
  <UserCard id={1} name="Fabio"></UserCard>
</div>

// revient à écrire
<div>
  <span />
  <UserCard id={1} name="Fabio"/>
</div>
```

Et certains mot-clés sont réservés en JavaScript:

```
// HTML
<label class="bigLabel" for="someInput"/>

// JSX
<label className="bigLabel" htmlFor="someInput"/>
```

Cela diffère donc de l'HTML.

CSS

Pour mettre en place le CSS sur React, on peut utiliser des fichiers CSS classiques:

✓ Avantages :

- Facile à mettre en place

✗ Inconvénients :

- Risque de conflits de noms
- Pas de scope local aux composants

```
/* styles.css */  
  
.button {  
  background-color: blue;  
}
```

```
import './styles.css';  
  
function Button() {  
  return <button className="button">Click  
me</button>;  
}
```


CSS Modules

On peut utiliser les modules CSS pour en limiter la portée:

✓ Avantages :

- Noms de classes localement scoped
- Support natif avec Vite

✗ Inconvénients :

- Non dynamique (conditions, thèmes, props etc.)

```
/* myComponent.module.css */  
  
.red {  
  color: red;  
}
```

```
import styles from './myComponent.module.css'  
  
const MyComponent = () => {  
  return (  
    <p className={styles.red}>It will be  
red</p>  
  )  
}
```

Inline CSS

On peut utiliser les modules CSS pour en limiter la portée:

✓ Avantages :

- Dynamique, conditionnel avec les props

✗ Inconvénients :

- Pas de pseudo-classes (:hover, etc.)
- Pas de media queries
- CSS mélangé à la logique

```
const Loader = ({ loading }) => {  
  const styles = {  
    opacity: loading ? '50%' : '100%'  
  }  
  return (  
    <p style={styles}>Lorem ipsum</p>  
  )  
}
```

CSS-in-JS

D'autres librairies existent pour faire du CSS-in-JS:

emotion

```
import { css } from '@emotion/css'

const color = 'blue'
const style = css`
  color: 'red';
  &:hover { color: ${color}; }
`

const SomeComponent = () => (
  <div css={style}> Hover to change
  color. </div>
)
```

styled-component

```
const Button = styled.a<{ $primary?:
boolean; }>`
  --accent-color: white;
  background: transparent;
  border: 1px solid var(--accent-color);
  color: var(--accent-color);
  transition: all 200ms ease-in-out;
  &:hover {
    filter: brightness(0.85);
  }
`
```

StyleX

```
import * as stylex from '@stylexjs/stylex';

const styles = stylex.create({
  base: { /*...*/ }
});

function CustomComponent({style}) {
  return <div {...stylex.props(styles.base, style)} />;
}
```

Tailwind

TailwindCSS est framework agnostic, très utilisé, et particulièrement performant.

```
const Button = () => {  
  return (  
    <button className="bg-blue-500 text-white px-4 py-2">  
      Click me  
    </button>  
  );  
}
```

✓ Avantages :

- Très rapide à écrire
- Pas de CSS à gérer séparément
- Fortement optimisé pour le build (purge, etc.)

✗ Inconvénients :

- HTML surchargé de classes
- Nécessite apprentissage initial

Props

Les props

Chaque composant que l'on crée accepte un et un seul argument: *props*. Cet argument sera toujours un objet.

Une *props* est une propriété que l'on passe du composant parent au composant enfant.

L'exemple suivant montre un composant utilisant la propriété *name*:

```
1 import React from 'react'
2
3 export const Welcome = (props) => {
4   return (
5     <p>
6       Bonjour {props.name}
7     </p>
8   )
9 }
```

Pour utiliser une variable javascript en jsx, on utilise les accolades.

Props

On peut passer plusieurs propriétés à un composant. Il suffit pour ça de spécifier un nom d'attribut (à gauche) suivi de sa valeur (à droite) (comme on le ferait en HTML).

```
<MyComponent propsA={valueA} propsB={valueC}/>
<UseCard name="Tony Stark" hero="Iron Man"/>
```

Pour accéder à ces propriétés, on peut le faire:

- via **this.props** dans les *class component*
- via l'argument **props** pour les *functional component*

⚠ Il ne faut jamais muter les valeurs passées par les *props* dans un composant.

Passer des props

On peut passer tous les types dans une props, comme ici une chaîne de caractères (string).

```
1 ReactDOM.render(  
2   <Welcome name="Fabio" />,  
3   document.getElementById('root')  
4 )
```

Mais on peut aussi passer une expression Javascript.

```
1 // Le résultat d'une fonction  
2 <Welcome name={"FABIO".toLowerCase()} />  
3 <Welcome id={33 + (25 * 100)} />  
4  
5 // Une variable, quel que soit le type (string, array, object, etc)  
6 const myName = "Fabio"  
7 <Welcome name={myName} />  
8  
9 // La référence d'une fonction  
10 <button onClick={() => { console.log("clicked !"); }} />
```


Props et Spread Operator

On peut également utiliser le spread operator lorsque l'on a plusieurs *props* à passer à un composant:

```
1  const user = {  
2    name: "Fabio",  
3    gender: "male"  
4  }  
5  
6  return (  
7    <Welcome {...user} />  
8  )
```

Equivalent à:

```
1  const user = {  
2    name: "Fabio",  
3    gender: "male"  
4  }  
5  
6  return (  
7    <Welcome name={user.name} gender={user.gender} />  
8  )
```

Props et valeurs par défaut

Si l'on passe une *props* sans valeur, celle-ci prendra la valeur *true* par défaut:

```
<SomeComponent visible />  
// est équivalent à  
<SomeComponent visible={true} />
```

Props.children

children est une propriété de l'objet props qui est toujours passée dans ce dernier. Elle contient les (éventuels) enfants du composant:

```
const DisplayThreeTimesComponent = (props) => {  
  return <div>  
    {props.children}  
    {props.children}  
    {props.children}  
  </div>  
}
```

```
const ParentComponent = () => {  
  return (  
    <div>  
      <Header />  
      <DisplayThreeTimesComponent>  
        <p>Hello !</p>  
      </DisplayThreeTimesComponent>  
    </div>  
  )  
}
```

Exercice: Passer des props

Cloner le répertoire suivant:

<https://github.com/FabioReact/exercice-react>

```
git clone https://github.com/FabioReact/exercice-react.git
cd exercice-react
npm install / npm i / yarn
npm run dev / yarn dev
```

Boucler

Voici comment boucler sur un itérable et de le retourner en JSX:

```
1  const names = ['Adam', 'Steve', 'Natasha']
2  const arrayOfLi = names.map((name) => <li>{name}</li>)
3  return (
4    <ul>
5      {arrayOfLi}
6    </ul>
7  )
8  // Ou encore
9  const names = ['Adam', 'Steve', 'Natasha']
10 return (
11   <ul>
12     {names.map(name => <li>{name}</li>)}
13   </ul>
14 )
```

Exercice sur les boucles

keys

Quand on crée des éléments dans une liste, typiquement en bouclant, React a besoin d'un identifiant unique (unique pour cette liste).

```
const names = ['Adam', 'Steve', 'Natasha']
return (
  <ul>
    {names.map((name, index) => <li key={index}>{name}</li>)}
  </ul>
)
```

React utilise ces identifiants pour savoir quel élément a disparu, apparu, ou a été changé.

Documentation: <https://fr.reactjs.org/docs/lists-and-keys.html>

Attention, dans cet exemple on utilise l'index du tableau comme identifiant unique... C'est une mauvaise pratique! Il faudrait utiliser un id vraiment unique (comme ceux de la base de données).

propTypes - sans TypeScript

Indique le type attendu de chacune des **props**

Est vérifié au runtime

```
import PropTypes from 'prop-types'

class Greeting extends React.Component {
  render() {
    return (
      <h1>Hello, {this.props.name}</h1>
    )
  }
}

Greeting.propTypes = {
  name: React.PropTypes.string
}
```

Beaucoup de types possibles:

<https://fr.reactjs.org/docs/typechecking-with-proptypes.html>

Alternative avec TypeScript

Indique le type attendu de chacune des **props**.

Si le type n'est pas correct, on aura des erreurs dans la console ainsi que dans notre IDE.

```
type Props = {  
  name: string;  
};  
  
const Greeting = (props: Props): JSX.Element => {  
  return (  
    <h1>Hello, {props.name}</h1>  
  );  
};
```


defaultProps

Valeur par défaut des props, pour la syntaxe avec **class**

```
class Welcome extends React.Component {  
  render() {  
    return (  
      <h1>Welcome, {this.props.name}</h1>  
    )  
  }  
}  
  
Welcome.defaultProps = {  
  name: "Fabio"  
}
```

Pour les *functional component*, il suffit simplement d'utiliser les features d'ES6!

```
const Greeting = ({name = "Fabio"}) => {  
  return <h1>Hello, {name}</h1>;  
}
```

Structure de notre application

Export

Afin qu'un composant soit utilisable ailleurs que dans le fichier où il a été créé, il faudra l'exporter.

Cependant, il est possible d'exporter une fonction / un composant de plusieurs manières:

Export par défaut:

```
const MyComponent = () => {  
  // ...  
}  
  
export default MyComponent
```

Export nommé (avec et sans *alias*):

```
const MyComponent = () => {}  
const MySecondComponent = () => {}  
  
export { MyComponent, MySecondComponent as MSComponent }
```

Il n'est possible de faire qu'un seul export par défaut par fichier, mais on peut faire autant d'export nommé que l'on souhaite:

```
export { fonction1 as default, fonction2, fonction3 }
```

Import

On devra ensuite importer la fonction / le composant conformément à la méthode utilisée pour l'exporter:

Import par défaut:

```
import MyComponent from "../path/to/component";
```

Il est utile de noter qu'on n'est pas obligé de conserver le nom d'origine grâce à l'import par défaut

```
import MonComposantRenomme from "../path/to/component";
```

Import nommé:

```
import { MyComponent } from "../path/to/component";
```

Je suis obligé d'importer le composant par son nom. Si je souhaite le renommer, je pourrais le faire via un *alias*.

```
import { MyComponent as MonComposantRenomme } from "../path/to/component";
```

Il est aussi possible de faire un import par défaut suivi d'imports nommés:

```
import fonctionParDefaut, { fonction2, fonction3 } from "../path/to/functions";
```

Structure de fichiers

React n'a pas d'opinion sur la structure de fichiers à adopter. Cette structure peut donc varier d'un.e projet/entreprise/équipe à un.e autre.

On peut cependant organiser son projet par fonctionnalité ou par type de fichiers.

```
src
├── hooks
│   └── useCustomHook.jsx
├── components
│   ├── Navbar.jsx
│   ├── Header.jsx
│   └── Footer.jsx
├── pages
│   ├── Home.jsx
│   └── Blog.jsx
└── api
    ├── posts.jsx
    └── comments.jsx
```

Data Flow - Données et événements

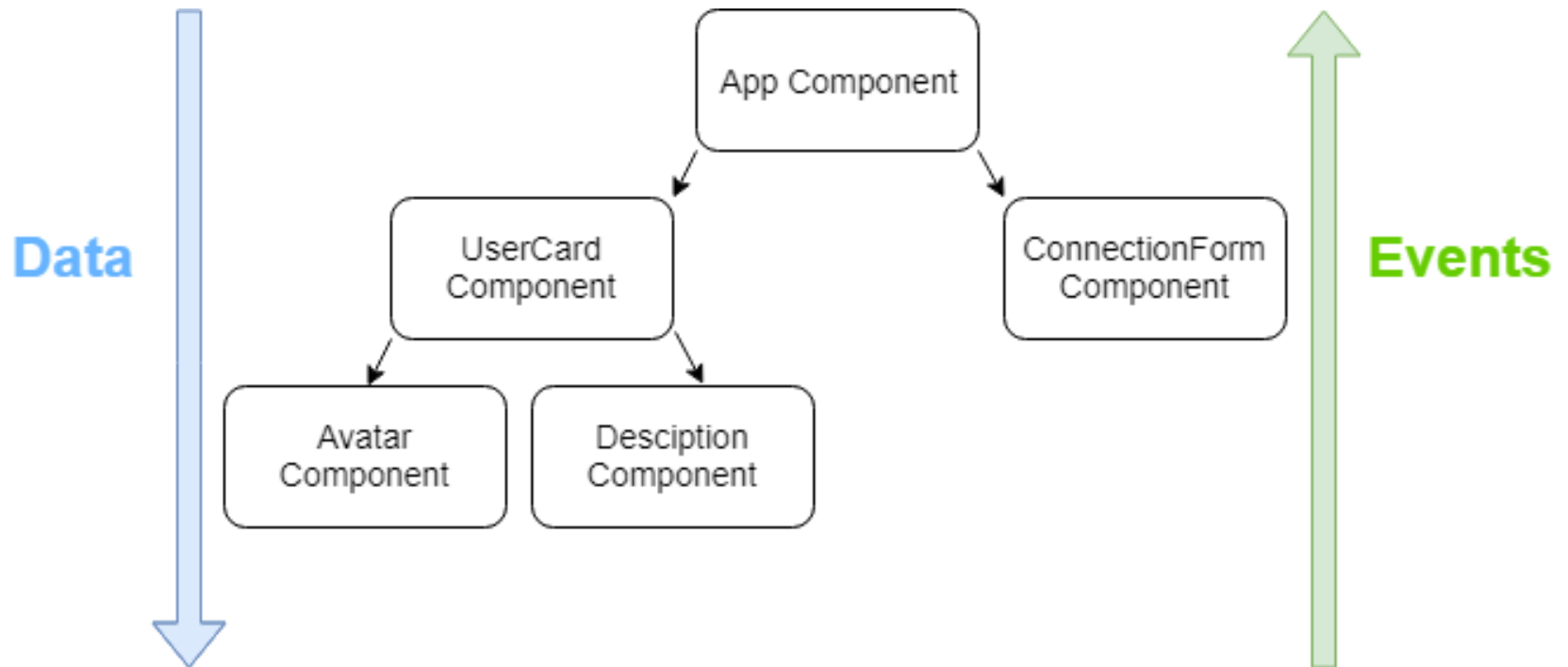
Data Flow

Les **props** ne vont que dans un seul sens : des composants parents vers les enfants.

Par conséquent, les composant enfants ne peuvent pas directement influencer l'affichage des parents.

Il existe cependant un moyen (indirect) de le faire:
les parents peuvent passer en props une référence à une fonction au composant enfant, fonction qui sera capable de modifier l'état du parent.

Data Flow



Data Flow

La plupart des Components de l'application vont être très simple.

On va avoir un ou quelques "container" components, à la racine de l'application, qui vont devoir se connecter d'une manière ou d'une autre au reste de l'application....

Dépend de la solution choisie : fait maison? Redux? React Context?

React Hooks

useState

Depuis React v16.8, les functional components peuvent aussi avoir un state:

```
import React, { useState } from 'react';

const Count = (props) => {
  const initialCount = 0; // Peut être un array [], un object {},
  // une string "chaine", un int...
  const [count, setCount] = useState(initialCount);
  return (<>
    Total : {count}
    <button onClick={() => setCount(initialCount)}>Réinitialiser</button>
    <button onClick={() => setCount(prevCount => prevCount + 1)}>+</button>
    <button onClick={() => setCount(prevCount => prevCount - 1)}>-</button>
  </>);
}

// Ici, utilisation de React.Fragment dans sa forme réduite: <> et </>
export default Count;
```

useState renvoie un tableau dont la première valeur est le state (getter), et la seconde une fonction (setter) permettant de modifier le state.

Attention, useState ne merge pas la nouvelle valeur avec les anciennes, utiliser le spread operator!

useEffect

useEffect est une alternative pour les functional component au lifecycle des class component. Il s'exécute à chaque render.

```
import React, { useEffect } from 'react';

const SomeComponent = (props) => {
  // Some code
  useEffect(() => {
    // Effect
    return () => {
      // Clean up effect
    };
  }, [input]);
  // Some Code
  return (
    //Some JSX
  )
}
```

Il prend deux paramètres:

1. Le premier est la fonction à exécuter lorsque celui-ci est déclenché
2. Le second un array de paramètre(s) déclencheur(s)

useLayoutEffect

useLayoutEffect a la même signature que *useEffect*. La différence entre ces derniers et le moment où les effets sont exécutés.

useLayoutEffect s'exécute de manière synchrone après une mise à jour et donc avant que le navigateur ne repeigne la page.

```
import React, { useLayoutEffect } from 'react';

const SomeComponent = (props) => {
  // Some code
  useLayoutEffect(() => {
    // LayoutEffect
    return () => {
      // Clean up effect
    };
  }, [input]);
  // Some Code
  return (
    //Some JSX
  )
}
```

useRef

useRef est l'alternative de *createRef* pour la création d'une référence dans un functional component.

```
import React, { useRef } from 'react';

const SomeComponent = (props) => {
  const usernameRef = useRef(null);
  const login = () => {
    const username = usernameRef.current.value;
    // Some code...
  }
  return (
    <>
      <input ref={usernameRef} type="text" id="username" />
      <button onClick={login}>Submit</button>
    </>
  )
};
```

L'objet retourné par *useRef* persistera pendant toute la durée de vie du composant. Les références sont utiles notamment pour accéder facilement au *DOM*.

useReducer

useReducer est une alternative de *useState*. Il est parfois plus aisé d'avoir un seul *reducer* plutôt qu'une multitude de *useState*.

```
import React, { useReducer } from 'react';

const reducer = (state, action) => {
  switch (action.type) {
    case 'increment':
      return {counter: state.counter + 1};
    case 'decrement':
      return {counter: state.counter - 1};
    default:
      throw new Error("Incorrect action type");
  }
}

const Counter = (props) => {
  const initialState = {counter: 0};
  const [state, dispatch] = useReducer(reducer, initialState);
  return (
    <>
      <p>Compteur: {state.counter}</p>
      <button onClick={() => dispatch({type: 'increment'})}>Increment</button>
      <button onClick={() => dispatch({type: 'decrement'})}>Decrement</button>
    </>
  )
};
```

useTransition

Depuis la version 18 de React, il est possible d'utiliser le *concurrent mode* afin de prioriser certaines mise à jour de l'UI. *useTransition* permet de spécifier des mises à jour de l'UI comme non urgente et qui puissent être interrompues. Cela est utile lorsque on a un affichage lent et qu'on puisse éventuellement différer (ex: un filtre).

```
import React, { useTransition } from 'react';

const MyComponent = (props) => {
  const [isPending, startTransition] = useTransition()

  const someEventHandler = (event) => {
    startTransition(() => {
      // Toutes les mises à jour faite ici sont déclarées comme non urgente et
      // peuvent être interrompues
      setValue(event.target.value);
    });
  }

  return (...)
```

```
};
```


useDeferredValue

useDeferredValue est similaire à *useTransition* mais à la différence qu'il s'utilise dans un composant enfant et dont les information proviennent du parent. Si une mise à jour plus urgente est faite, React retournera la valeur précédente et ne mettra à jour que par la suite les mises à jour non urgentes.

```
import React, { useDeferredValue } from 'react';

const MyComponent = ({ list }) => {
  const deferredValue = useDeferredValue(list);

  return (
    <>{deferredValue.map(el => <OtherComponent info={el} />)}</>
  )
};
```

Pour tirer tout les avantages de *useDeferredValue* il faudra également penser à mémoriser notre composant.

Conditional Rendering

Affichage Conditionnel

Il est possible d'afficher un composant ou non en fonction d'une condition donnée.

La première solution est d'utiliser une condition if:

```
function HelloUser(props) {  
  if (!props.name) return (<p>Hello visitor</p>)  
  return (<p>Hello {props.name}</p>)  
}
```

Ou une condition ternaire:

```
function HelloUser(props) {  
  return (<p>Hello {props.name ? props.name: 'visitor'}</p>)  
}
```

Ou stocker le composant dans une variable:

```
function HelloUser(props) {  
  let component = null  
  if (props.name) component = (<p>Hello {props.name}</p>)  
  else component = (<p>Hello visitor</p>)  
  
  return component  
}
```

&& - ||

L'opérateur logique && permet de renvoyer ce qui sera à droite de ce dernier si ce qui est à gauche est *truthy*:

```
function HelloUser(props) {  
  return (  
    <p>Hello gamer</p>  
    {props.points && <p>You have {props.points} points so far. Good job!</p>}  
  )  
}
```

L'opérateur logique || permet de renvoyer le premier élément qui sera *truthy*:

```
function HelloUser(props) {  
  return (  
    <p>Hello gamer</p>  
    <p>You have {props.points || 0} points so far.</p>  
  )  
}
```

Formulaires

Forms

Dans un formulaire, on peut spécifier la *value* d'un input. On appelle alors cela un **Controlled** input.

Pour ce faire, on spécifie la propriété *value*.

```
<input  
  type="text"  
  value="My value"  
>
```

Mais du coup, comment celle-ci peut changer?

Forms (Controlled)

On doit donc écouter manuellement les changements, les appliquer au *state*, et utiliser le *state* pour la *value*.

```
constructor(props) {  
  super(props);  
  this.state = {inputValue: "My default value"};  
  this.handleChange = this.handleChange.bind(this);  
  this.handleSubmit = this.handleSubmit.bind(this);  
}  
handleChange(event) {  
  this.setState({inputValue: event.target.value});  
}  
handleSubmit(event) {  
  console.log('The submitted value was ' + this.state.inputValue);  
  event.preventDefault();  
}  
render() {  
  return (  
    <form onSubmit={this.handleSubmit}>  
      <input  
        type="text"  
        value={this.state.inputValue}  
        onChange={this.handleChange}  
      />  
      <button>OK</button>  
    </form>  
  );  
}
```

Forms (Controlled) - Hooks

On doit donc écouter manuellement les changements, les appliquer au *state*, et utiliser le *state* pour la *value*.

```
const [inputValue, setInputValue] = useState('')

const handleChange = (event) => {
  setInputValue(event.target.value)
}

const handleSubmit = (event) => {
  event.preventDefault()
  console.log('The submitted value was ' + inputValue)
}

return (
  <form onSubmit={handleSubmit}>
    <input
      type="text"
      value={inputValue}
      onChange={handleChange}
    />
    <button>OK</button>
  </form>
);
```


Forms (Uncontrolled)

On ne spécifie pas la prop *value* (on peut juste spécifier *defaultValue*), et on utilise **ref** pour récupérer la valeur.

```
constructor(props) {  
  super(props);  
  this.inputRef = React.createRef();  
}  
  
handleSubmit(event) {  
  console.log('we got the value ' + this.inputRef.current.value);  
  event.preventDefault();  
}  
  
render() {  
  return (  
    <form onSubmit={this.handleSubmit.bind(this)}>  
      <input  
        type="text"  
        defaultValue="A default value"  
        ref={this.inputRef} />  
      <button>OK</button>  
    </form>  
  );  
}
```

Forms: Quelle option choisir?

feature	uncontrolled	controlled
one-time value retrieval (e.g. on submit)	✓	✓
validating on submit	✓	✓
instant field validation	✗	✓
conditionally disabling submit button	✗	✓
enforcing input format	✗	✓
several inputs for one piece of data	✗	✓
dynamic inputs	✗	✓

Plusieurs bibliothèques existent pour nous aider à gérer les formulaires (React Hook Form étant la plus utilisée).

Optimisations

React Lazy

Permet d'afficher du JSX ou un autre component en attendant le chargement du 'lazy' component.

```
import React, { Suspense, lazy } from 'react';
import Profile from './components/Profile';

const User = lazy(() => import('./components/User'));

const App = () => (
  <React.Fragment>
    <Profile />
    <Suspense fallback={<div>Loading...</div>}>
      <User />
    </Suspense>
  </React.Fragment>
);
```

Pure Component - Classes

Pure Components est une alternative à l'utilisation de la méthode *shouldComponentUpdate* des class components.

```
import React, { PureComponent } from 'react';

class SomeComponent extends PureComponent {
  // Some Code
  render() {
    return (
      // Some JSX
    );
  }
};
```

Ce dernier effectue un *shallow check* du state et des props. S'il n'y a eu aucun changement, le composant ne sera pas mis à jour.

Très efficace pour éviter des rendu inutiles.

N'utiliser que *PureComponent* si les props ou le state du parent peuvent vraiment provoquer un rendu inutile du component enfant.

React Memo

React Memo est une alternative à l'utilisation de la méthode `shouldComponentUpdate` pour les fonctional components.

```
import React from 'react';

const SomeComponent = (props) => {
  // Some Code
  return (
    // Some JSX
  );
};

export default React.memo(SomeComponent);
```

Même objectif que les `PureComponent`.

Attention, n'effectue qu'une *shallow* comparaison et peut ne pas détecter des changements à l'intérieur d'un tableau, objet (reference types).

useCallback

useCallback permet de renvoyer une fonction de rappel mémorisée. Pour cela on doit passer la fonction que l'on souhaite mémoriser, ainsi qu'un tableau de dépendance indiquant les entrées de notre fonction:

```
import React, { useCallback } from 'react';

const SomeComponent = (props) => {
  // ...
  const memoizedCallback = useCallback(
    () => {
      doSomething(a, b);
    },
    [a, b],
  );
  // ...
};
```

Cela permet d'éviter des rendu inutiles lorsqu'on passe une fonction à un composant utilisant *shouldComponentUpdate*, *React.memo*, ou *PureComponent*.

useMemo

useMemo permet de renvoyer une valeur mémorisée. Pour cela on doit passer la fonction dont on souhaite obtenir le résultat, ainsi qu'un tableau de dépendance indiquant les entrées de notre fonction:

```
import React, { useMemo } from 'react';

const SomeComponent = (props) => {
  // ...
  const memoizedValue = useMemo(() => computeExpensiveValue(a, b), [a, b]);
  // ...
};
```

Si les inputs n'ont pas changé, alors on gardera la valeur précédente du retour de la fonction.

Cela est très utile lorsqu'on fait appel à une fonction qui fait des calcul coûteux.

Si on ne fournis aucun tableau, une nouvelle valeur sera calculée à chaque appel.

Unit Test

Installation de vitest

Afin de faire mettre en place des test unitaires sur un projet React mis en place avec Vite, on va devoir installer Vitest,

```
npm install -D vitest
```

Une fois cette dépendance ajoutée, on peut ajouter à nos scripts:

```
1 {  
2   ...  
3   "scripts": {  
4     "dev": "vite",  
5     "build": "vite build",  
6     "test": "vitest"  
7   },  
8   ...  
9 }
```

Configuration

On ajoute ensuite ceci au fichier [vite.config.ts](#) afin de créer notre configuration de test:

```
/// <reference types="vitest/config" />

import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

// https://vitejs.dev/config/
export default defineConfig({
  plugins: [react()],
  test: {
    globals: true,
    environment: 'jsdom',
  },
});
```

On ajoute aussi dans notre fichier [tsconfig.app.json](#) les *globals* de vite:

```
{
  "compilerOptions": {
    "types": ["vite/client", "vitest/globals"],
  }
}
```

Premier fichier de test

On peut maintenant écrire notre premier test.

describe permet de créer un bloc qui regroupe plusieurs tests liés.

it, ou *test* décrit un test. Le premier argument décrit ce qu'on cherche à tester, et le second est la fonction qui contient nos assertions.

```
describe('something truthy and falsy', () => {  
  it('should be true', () => {  
    expect(true).toBe(true);  
  });  
  
  test('false to be false', () => {  
    expect(false).toBe(false);  
  });  
});
```

Installation de testing-library

On va enfin installer [testing library](#) qui nous permet de tester une application React

```
npm install --save-dev @testing-library/react @testing-library/dom @testing-library/jest-dom @types/react @types/react-dom
```

On ajoute de nouveau dans notre [tsconfig.app.json](#) les types de *jest-dom*:

```
{
  "compilerOptions": {
    "types": ["vite/client", "vitest/globals", "@testing-library/jest-dom"],
  }
}
```

Création d'un fichier de test

On peut maintenant utiliser les fonction de testing librabby pour tester notre composant:

```
import { render, screen } from '@testing-library/react'
import userEvent from '@testing-library/user-event'
import Fetch from './fetch'

test('loads and displays greeting', async () => {
  // render permet de charger notre composant dans notre test
  render(<Fetch url="/greeting" />)

  // userEvent permet de simuler des actions utilisateur
  await userEvent.click(screen.getByText('Load Greeting'))
  // getBy throw une erreur si le noeud n'est pas trouvé contrairement à findBy
  await screen.findByRole('heading')

  // ASSERT
  expect(screen.getByRole('heading')).toHaveTextContent('hello there')
  expect(screen.getByRole('button')).toBeDisabled()
})
```

react-testing-library

Les *queries* sont les méthodes qui vont nous permettre de trouver nos éléments sur la page HTML

	No match	1 match	1+ Match	Await
getBy	throw	return	throw	No
findBy	throw	return	throw	Yes
queryBy	null	return	throw	No
getAllBy	throw	array	array	No
findAllBy	throw	array	array	Yes
queryAllBy	[]	array	array	No

React Context

React Context

Le contexte sert à partager une valeur entre différents composants quel que soit la distance entre ces derniers.

La première étape est de créer un contexte.

```
// theme-context.js
import { createContext } from 'react';

const ThemeContext = createContext({
  foreground: "#000000",
  background: "#ffffff"
});

export default ThemeContext;
```

Les valeurs renseignées dans la méthode `createContext` ne sont pas des valeurs par défaut. Celles-ci seront néanmoins utiles pour de l'autocomplétion lors de l'utilisation du contexte.

Vous pouvez créer autant de contexte différent que vous le souhaitez.

React Context - Provider

La deuxième étape sera de préciser quelle partie de l'application aura accès à ce contexte. Seuls les enfants de la balise *Provider* aura accès ensuite au contexte donné.

```
// App.jsx
const App = () => {
  return (
    <>
      <ThemeContext.Provider value={{
        foreground: "#ffffff",
        background: "#222222",
      }}>
        <Home />
      </ThemeContext.Provider>
      <Footer />
    </>
  );
}
```

Dans cet exemple, le footer n'aura pas accès au contexte du thème.

React Context - useContext

La dernière étape sera de consommer notre contexte. Il existe deux façons de le faire. Cette dernière reste la plus simple:

```
// ThemeButton.jsx
import ThemeContext from "../context/theme-context";

const ThemedButton = () => {
  const theme = useContext(ThemeContext);
  return (
    <button style={{ background: theme.background, color: theme.foreground }}>
      Je suis stylé par le contexte de thème !
    </button>
  );
}

export default ThemeContext;
```

Ou encore de créer un custom hook:

```
import ThemeContext from "../context/theme-context";

const useTheme = () => useContext(ThemeContext);

export default useTheme;
```

React Context - Consumer

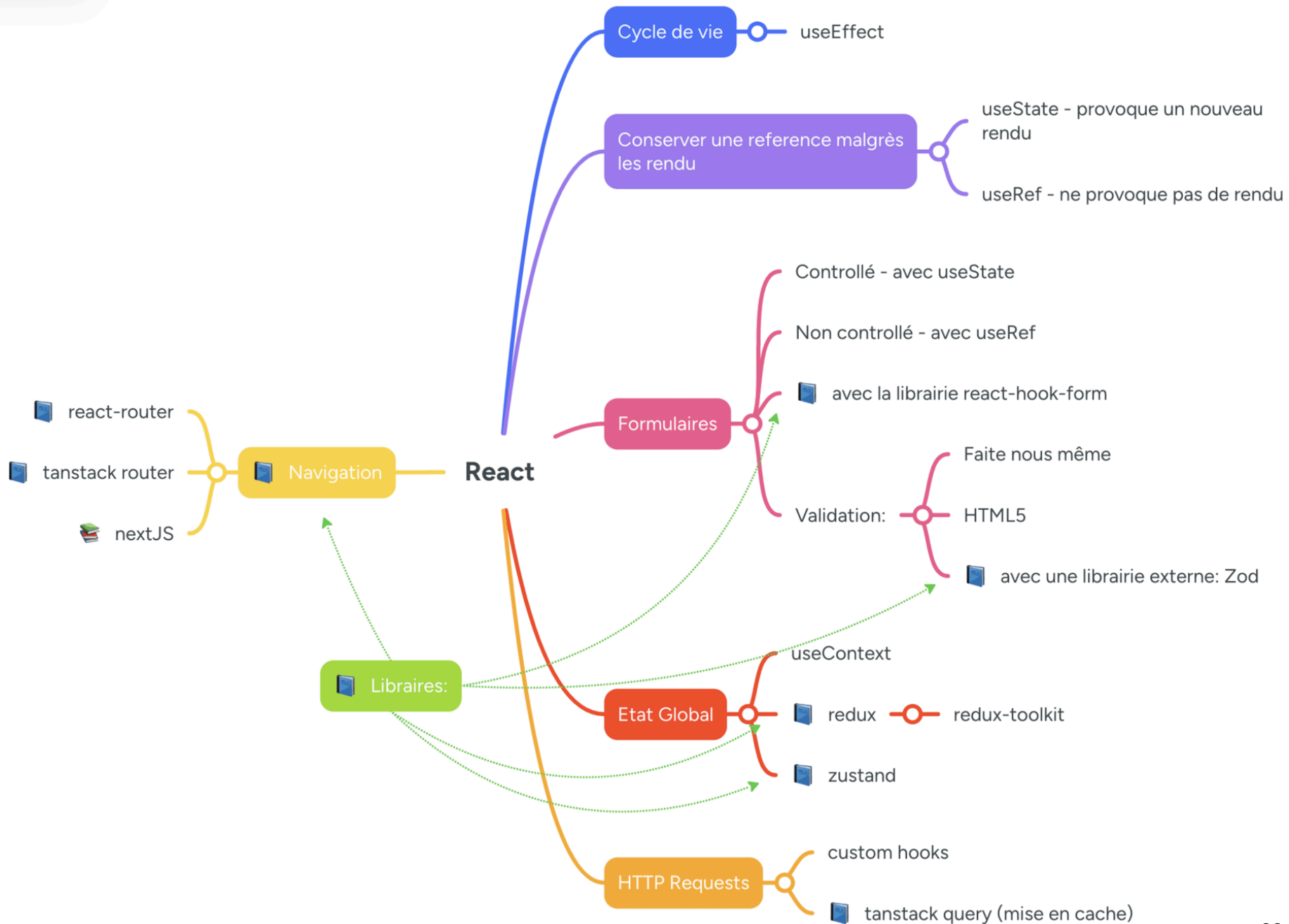
On peut aussi consommer le contexte via la balise *Consumer* comme suit:

```
// ThemeButton.jsx
import ThemeContext from "../context/theme-context";

const ThemedButton = () => {
  return (
    <ThemeContext.Consumer>
      {(theme) => (
        <button style={{
          background: theme.background,
          color: theme.foreground
        }}>
          Je suis stylé par le contexte de thème !
        </button>
      )}
    </ThemeContext.Consumer>
  );
}

export default ThemeContext;
```

Carte Mentale



Redux - Concepts

Principes

Les 3 principes de redux sont les suivants:

Single source of truth

L'état de toute notre application est stocké dans un seul objet *store*.

State is read-only.

Le seul moyen de changer le state est via l'émission (*dispatch*) d'une *action*. Une *action* décrit ce qu'il vient de se passer.

Changes are made with pure functions

L'état de notre application (*store*) sera changé à partir de fonctions pures: les *reducers*.

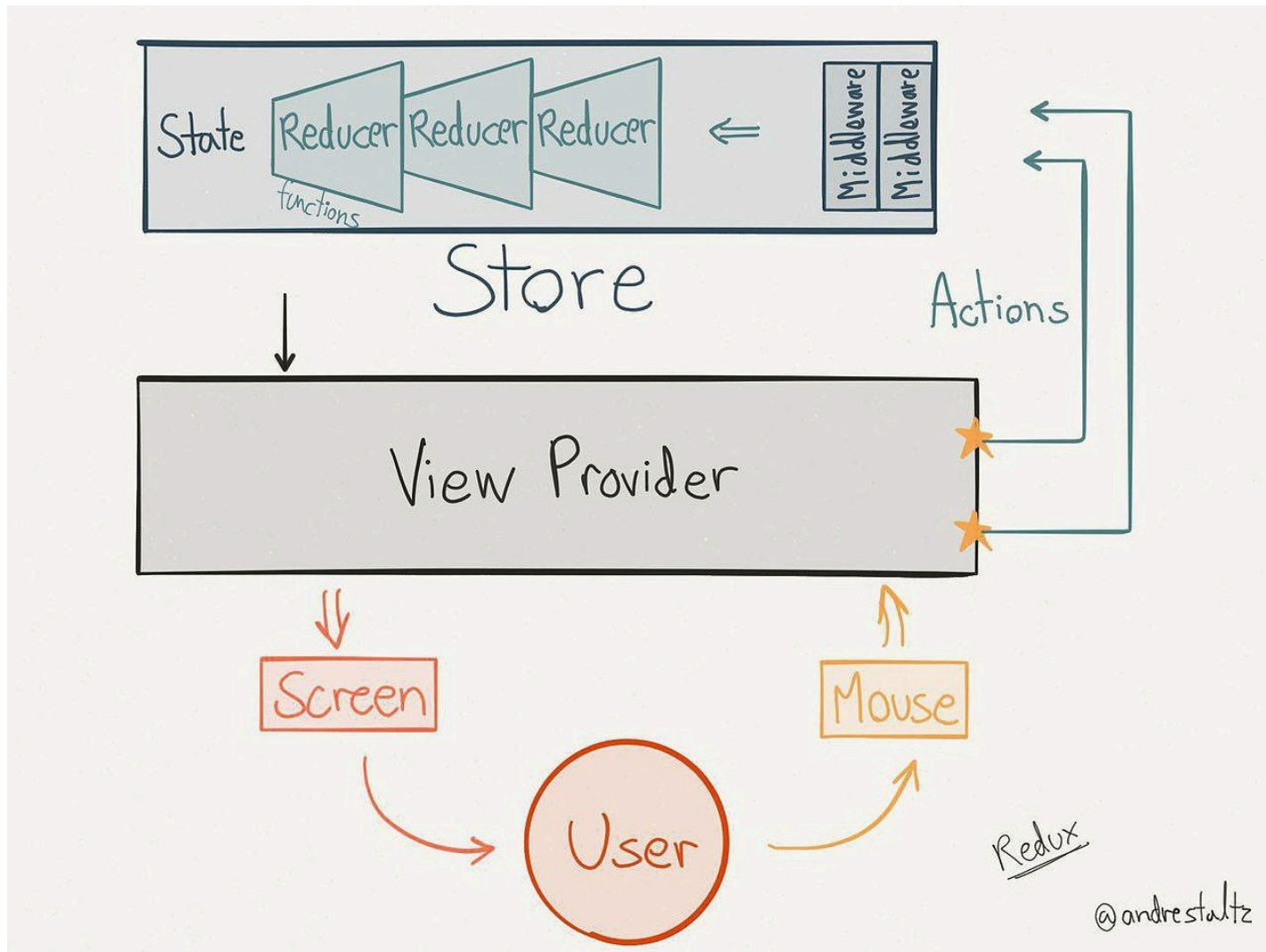
Une fonction *pure* retourne toujours le même output pour un input donné, et ne doit pas avoir d'effets de bord.

Fonctionnement

Le *store* contient l'état de notre application. Pour changer cette état, on va *dispatch* des *actions*. Ces *actions* vont être traités par un *reducer* qui va déterminer l'état futur de notre application en fonction de l'*action* qui lui a été transmise.



Fonctionnement



Store

Le *store* (état de notre application) est un simple objet javascript:

```
{
  todos: [{
    text: 'Eat food',
    completed: true
  }, {
    text: 'Exercise',
    completed: false
  }],
  visibilityFilter: 'SHOW_COMPLETED'
}
```

Actions

Les *actions* sont également des objets javascript. Cependant celles-ci doivent **impérativement** contenir la propriété *type* (ce dernier étant une *string*).

```
{ type: 'ADD_TODO', text: 'Go to swimming pool' }  
{ type: 'TOGGLE_TODO', index: 1 }  
{ type: 'SET_VISIBILITY_FILTER', filter: 'SHOW_ALL' }
```

D'autres propriétés peuvent lui être ajoutés, elles sont optionnelles dans l'absolu mais peuvent être nécessaire selon le traitement que vous en faîte dans le *reducer*.

Actions Creators

Les *actions creators* sont des fonctions qui retournent une action.

```
function addTodo(text) {  
  return {  
    type: ADD_TODO,  
    text  
  };  
}
```

Les *actions creators* vont notamment nous être utile pour la mise en place de requêtes asynchrones.

Reducer

Un *reducer* est une fonction qui a la spécificité de devoir être *pure*.

```
const initialState = {};  
  
function rootReducer(state = initialState, action) {  
  // On mettra en place ici un switch/case pour une faire un traitement particul.  
  // selon le type de l'action qui aura été passé en argument lors du dispatch  
  // et ainsi retourner le nouvel état de l'application  
  return state  
};
```

Le *reducer* doit prendre comme premier argument le *state* courant (qu'il faudra initialiser au départ) et pour second l'*action* qui lui sera *dispatch*.

Redux avec React (legacy)

Packages

Pour installer redux dans notre application react, il nous faudra installer les deux packages suivant:

redux et react-redux

```
npm install --save redux react-redux  
OU  
yarn add redux react-redux
```


Le Provider

On souhaite que toute notre application react ai accès à redux, pour cela on va mettre notre composant *Provider* (fournit par *react-redux*) à la racine de notre application:

```
// src/index.js
import { createStore } from 'redux';
import { Provider } from 'react-redux';
import rootReducer from './store/reducer';

const myStore = createStore(rootReducer);

ReactDOM.render(
  <Provider store={myStore}>
    <App />
  </Provider>,
  document.getElementById('root')
);
```

Pour créer notre *store*, on fait appel à la fonction *createStore* de *redux*. Pour ce faire on doit passer en paramètre à *createStore* notre *reducer*.

Le Reducer

```
// src/store/reducer.js
import * as actionTypes from './actionTypes'

const initialState = {
  todos: []
};

export const reducer = (state = initialState, action) => {
  switch (action.type) {
    case actionTypes.ADD_TODO:
      return {
        ...state,
        todos: [
          ...state.todos,
          {
            text: action.text,
            completed: false
          }
        ]
      }
    default:
      return state
  }
}
```

Ici, on importe les action types d'un autre fichier js. Il faut alors le créer.

Les Action types

Par convention, les types des *actions* sont en uppercase. Le fait de stocker nos types d'*actions* dans une variable permet de générer des erreurs lors de la compilation si on fait une faute de frappe (lors de la création d'une *action* ou lors de sa lecture dans le *reducer*).

```
// src/store/actionTypes.js
export const ADD_TODO = "ADD_TODO"
```

```
// src/store/actionCreators.js
import * as actionTypes from './actionTypes'
export const addTodo = (text) => {
  return {
    type: actionTypes.ADD_TODO,
    text: text
  }
}
```

A partir de cette étape, on a créé avec succès notre store. On va pouvoir désormais connecter nos composants avec le store.

Connecter nos composants

On va maintenant voir comment "connecter" un de nos composants avec redux. Une fois un composant connecté, il recevra via ses *props* ce que l'on aura déterminé.

On recevra deux types de *props*:

- des données provenant du *store*, que l'on va pouvoir récupérer via la fonction *mapStateToProps* (va injecter dans nos *props* le *state* que l'on aura précisé de récupérer)
- des fonctions *callback*, pour la mise à jour du store, que l'on pourra récupérer via *mapDispatchToProps*

Connecter nos composants

Afin de connecter notre composant, il faut procéder comme suit:

```
// src/pages/ToDoComponent.js
import React from "react"
import { connect } from "react-redux"
import { addToDo } from "../store/actionCreators.js"

const ToDoComponent = props => {
  // ...
}
// mapStateToProps reçoit comme paramètre le store actuel de redux
const mapStateToProps = state => {
  return {
    todos: state.todos // props.todos (type []) sera accessible dans notre
    component
  }
}
// mapStateToProps reçoit comme paramètre la fonction dispatch de redux
const mapDispatchToProps = dispatch => {
  return {
    onAddToDo: (text) => dispatch(addToDo(text)) // props.onaddToDo (type fn)
    sera aussi accessible
  }
}
// connect permet de connecter ce que l'on vient de définir à notre composant
export default connect(mapStateToProps, mapDispatchToProps)(ToDoComponent)
```

Redux DevTools

Redux DevTools est une extension que l'on peut installer sur son navigateur afin d'avoir des outils de développement/debug.

1. Installer l'extension sur votre navigateur
(<https://github.com/zalmoxisus/redux-devtools-extension>)
2. Modifier l'*index.js*

```
import { createStore, applyMiddleware, compose } from 'redux';
import reducer from './reducer';
import someMiddleware from './someMiddleware';

const composeEnhancers = window.__REDUX_DEVTOOLS_EXTENSION_COMPOSE__ ||
compose;

const store = createStore(
  reducer,
  composeEnhancers(applyMiddleware(someMiddleware))
)
```

Redux asynchrone (legacy)

Asynchrone et redux-thunk

Jusqu'ici, tout est *synchrone*. Les *dispatch* font appel au *reducer* et ce dernier doit être *pure*...

Comment faire donc de l'*asynchrone* afin de faire un appel à la base de données?

Dès que l'on travaille avec un API, on enfreint les règles d'une fonction pure car:

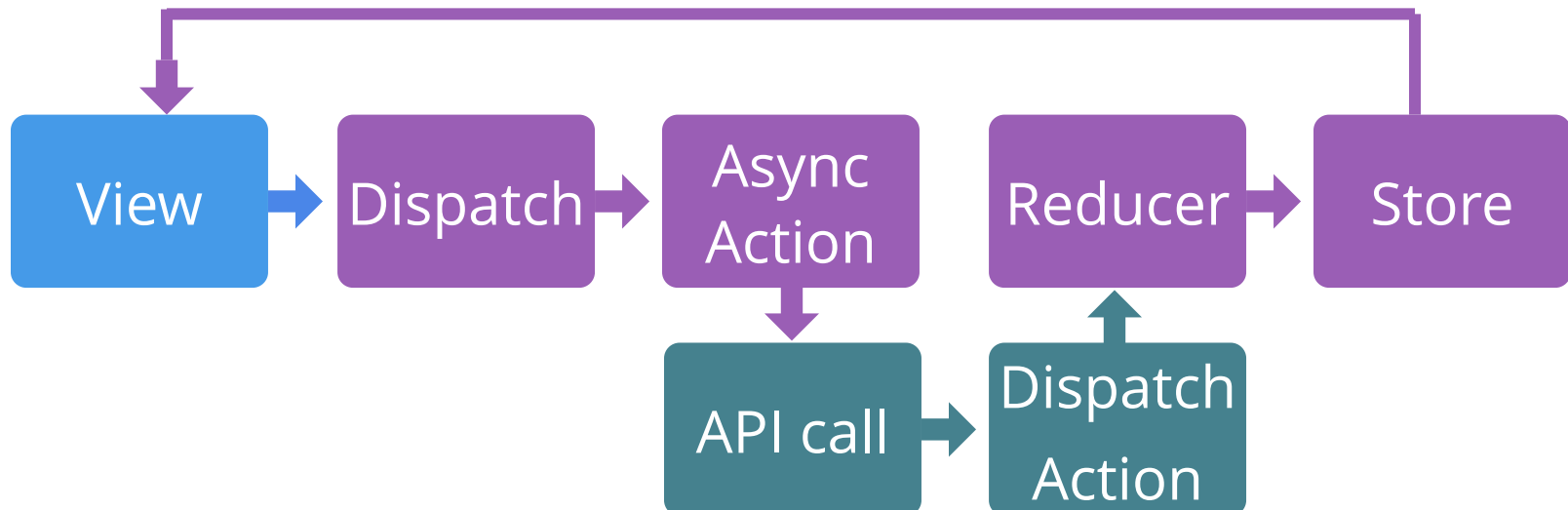
- on a un effet de bord
- notre réponse est forcément asynchrone

Pour résoudre notre problème, on va utiliser le *middleware: redux-thunk*

```
npm install --save redux-thunk  
OU  
yarn add redux-thunk
```


redux-thunk

redux-thunk est une *middleware* qui va se positionner entre le *dispatch* de l'*action* et notre *reducer*, ce qui lui permet d'effectuer un traitement particulier. On va faire ça en dispatchant un nouvel *action creator* qui pourra être *asynchrone* grâce à *redux-thunk*.



Mise en place

Une fois notre package `redux-thunk` installé, on va le mettre en place sur notre projet:

```
1 // src/index.js
2 import { createStore, applyMiddleware, compose } from 'redux';
3 import thunkMiddleware from 'redux-thunk'
4 import { Provider } from 'react-redux';
5 import rootReducer from './store/reducer';
6
7 const withMiddleware = applyMiddleware(thunkMiddleware)
8 const myStore = createStore(rootReducer, undefined,
  compose(withMiddleware));
9 ReactDOM.render(
10   <Provider store={myStore}>
11     <App />
12   </Provider>,
13   document.getElementById('root')
14 );
```

On fait appel à notre *thunkMiddleware*, et on utilise la fonction *applyMiddleware* de *redux* afin de l'appliquer, *applyMiddleware* peut prendre autant de *middleware* en arguments que besoin.

compose de *redux* est une fonction afin de composer avec nos (possibles) différents *enhancers* (*withMiddleware* est un *enhancer*).

Async Action types

On peut maintenant créer des *action creators* asynchrones. Ces derniers pourront par la suite appeler des *action creators* à travers la fonction *dispatch*.

```
// src/store/actionTypes.js
export const ADD_TODO = "ADD_TODO"
```

```
// src/store/actionCreators.js
import * as actionTypes from './actionTypes'
export const addToDo = (text) => {
  return {
    type: actionTypes.ADD_TODO,
    text: text
  }
}

export const asyncAddToDo = (text) => {
  return dispatch => {
    // Exécuter de l'asynchrone ici...
    await response = fetch("URL")
    dispatch(addToDo(text))
  }
}
```

Dans notre composant...

Le composant devra désormais uniquement appelé notre *action creator* asynchrone.

```
1 // src/pages/ToDoComponent.js
2 import { asyncAddToDo } from "../store/actionCreators.js"
3
4 const mapDispatchToProps = dispatch => {
5   return {
6     onAddToDo: (text) => dispatch(asyncAddToDo(text))
7   }
8 }
9
10 export default connect(mapStateToProps, mapDispatchToProps)(ToDoComponent)
```

Class Components (Déprécié)

Première Syntaxe: Class

class App extends React.Component

```
import React from 'react'

class MyComponent extends React.Component {
  constructor(props) {
    super(props);
    // ...
  }

  someCustomMethod() {
    // Code
  }

  render() {
    return (<div>
      // Some Content
    </div>)
  }
}
```

Ou encore

```
import React, { Component } from 'react'

class MyComponent extends Component {
  // ...
}
```

Seconde Syntaxe:

Les "*functional components*", la syntaxe est plus succincte, mais son comportement est cependant différent (cycle de vie). Se résume à écrire la méthode *render()* en prenant les *props* comme argument.

```
const MyComponent = (props) => {  
  return (  
    <div>  
      { (...) }  
    </div>  
  );  
}
```

Quelle syntaxe choisir?

Class Component vs Functional Component

Class Component

`class Learn extends Component`

- Possède un state
- Accès au cycle de vie

Utilise "this" pour accéder au state/props:

`this.state.someVariable`

`this.props.someVariable`

Functional Component

`const Learn = () => {...}`

- Possède un state depuis v16.8
- Cycle de vie depuis v16.8

Accède directement aux props:

`someState`

`props.someVariable`

Utilisez une classe lorsque vous avez besoin d'un state / lifecycle hooks et que vous ne pouvez pas utiliser les React Hooks (v16.8), sinon, utilisez un Functional component.

State

Le **state** est un objet qui représente un état interne du composant.

Lorsque une valeur dans le state est "mise à jour", alors l'application de met à jour.

Un usage serait un composant dont l'apparence changerait en fonction d'événements internes, et qui n'impacte pas le fonctionnement des composants parents.

Exemple: un carrousel

this.state

Il est uniquement possible d'y accéder de cette manière dans la syntaxe de classe.

this.state.someProperty pour lire les valeurs

this.setState({ someProperty : "something "}) pour setter les valeurs

this.setState() merge intelligemment (mais shallow) avec le valeurs courantes, pas besoin de tout redéfinir.

On ne doit **jamais** setter de cette de cette façon:

~~**this.state = (...);**~~

...sauf dans le constructeur (phase d'initialisation du state) !

this.state

```
class ToggleableImage extends React.Component {  
  constructor(props) {  
    super(props);  
    this.state = {displayImage: true};  
  }  
  
  toggleImage() {  
    this.setState({  
      displayImage : !this.state.displayImage  
    });  
  }  
  
  render() {  
    return (  
      <div>  
        <button onClick={this.toggleImage.bind(this)}>Show/hide the image</button>  
        {this.state.displayImage && <img src={logo}/>}  
      </div>  
    );  
  }  
}
```

Attention, dans cet exemple displayImage prend la valeur inverse de this.state.displayImage. Cette façon de procéder n'est pas correcte, on vas voir pourquoi...

this.setState()

Les **setState()** étant asynchrones, on n'a aucune certitude que la valeur précédente est bien celle à laquelle on s'attend.

```
// Peut se tromper
this.setState({
  counter: this.state.counter + 1
})

// Syntaxe toujours correcte !
// Reçoit le state courant et les props courantes au moment
// où l'update sera appliqué
this.setState((prevState, props) => ({
  counter: prevState.counter + 1
}))
```

Attention donc si vous dépendez des props courantes ou du state courant.

Attacher des events handler

Problème : perte du *this*

```
class SayHello extends React.Component {  
  
  handleClick() {  
    // KO : "this" ici n'est pas la classe !  
    this.setState({})  
  }  
  
  render() {  
    return (  
      <button onClick={this.handleClick}>  
        Say hello  
      </button>  
    );  
  }  
}
```

Attacher des events handler

Solution 1 : *.bind(this)* à chaque usage

```
class SayHello extends React.Component {  
  
  handleClick() {  
    // Ok  
    this.setState({})  
  }  
  
  render() {  
    // .bind(this) force à transmettre le this  
    return (  
      <button onClick={this.handleClick.bind(this)}>  
        Say hello  
      </button>  
    );  
  }  
}
```

Non optimale car une fonction supplémentaire est appelée à chaque clique.

Attacher des events handler

Solution 2 : *.bind(this)* dans le *constructor* (la plus courante)

```
class SayHello extends React.Component {  
  
  constructor(props) {  
    super(props);  
    // on bind à this et on écrase  
    this.handleClick = this.handleClick.bind(this);  
  }  
  
  handleClick() {  
    // OK  
    this.setState(...)  
  }  
  
  render() {  
    return (  
      <button onClick={this.handleClick}>  
        Say hello  
      </button>  
    );  
  }  
}
```

Attacher des events handler

Solution 3 : utiliser une arrow function (pas de changement de contexte avec cette dernière)

```
class SayHello extends React.Component {  
  
  constructor(props) {  
    super(props);  
  }  
  
  handleClick = () => {  
    // OK  
    this.setState({});  
  }  
  
  render() {  
    return (  
      <button onClick={this.handleClick}>  
        Say hello  
      </button>  
    );  
  }  
}
```


refs

Une ref nous permet d'obtenir une référence vers un noeud du DOM du composant, afin d'en faire un usage spécifique.

```
constructor(props) {  
  super(props);  
  this.myRef = React.createRef();  
}  
  
componentDidMount() {  
  // création d'une chart avec ChartJS  
  new Chart(this.myRef.current, {  
    (...)  
  });  
}  
  
render() {  
  // On utilise le `ref` callback pour stocker la référence vers  
  // le noeud DOM dans un champ custom du this  
  return (  
    <div ref={this.myRef} />  
  );  
}
```

Class Component Lifecycle (Déprécié)

Introduction

On peut diviser le cycle de vie d'une composant en 3 étapes successives:

1. Création (Mount)
2. Mise à jour (Update)
3. Destruction (Unmount)

Documentation: <http://projects.wojtekmaj.pl/react-lifecycle-methods-diagram/>

Création (Mount)

Phase d'initialisation du composant.

A ce stade, ni les *props*, ni le *state* ne sont encore définis.

Le composant ainsi que tous ses enfants sont montés dans l'UI.

Création (Mount)

```
class MyComponent extends Component {  
  constructor(props) {  
    super(props)  
    this.state = {}  
  }  
  
  static getDerivedStateFromProps(nextProps,  
prevState)  
  {  
    if (true) {  
      return someNewState  
    }  
    return null  
  }  
  
  render() {  
    // JSX  
  }  
  
  componentDidMount() {  
    // Code After-render  
  }  
}
```

initialisation du state

met à jour le state en
fonction des props

rafraîchit l'UI

Le composant est
désormais monté, on
peut maintenant faire
des requêtes à la bdd

Mise à jour (Update)

```
class MyComponent extends Component {  
  shouldComponentUpdate(nextProps, nextState) {  
    // Renvoie "true" par défaut  
  }  
}
```

Retourne un booléen,
permet d'éviter le
rafraîchissement inutile
du composant

```
  getSnapshotBeforeUpdate(prevProps, prevState) {  
    // Utilise si on a besoin de faire  
    // des calculs selon l'état précédent  
  }
```

permet de capturer des
infos du DOM courant

```
  componentDidUpdate() {  
    // Après le re-render  
  }
```

appelée après que le
nouveau render ai eu
lieu

```
  render() {  
    // re-render  
  }
```

rafraîchit l'UI

```
}
```

Destruction (Unmount)

```
class MyComponent extends Component {  
  componentWillUnmount() {  
    // Code unsubscribe  
  }  
}
```

Le composant n'est plus dans l'UI, endroit idéal pour se désabonner à d'éventuels événements.

Attention!

Si vous ne le faites pas, vous vous exposez à des fuites mémoires.